

FEMAssembler::buildAffineSystem() 数学模型与实现解读

独立推导说明

2026 年 5 月 27 日

摘要

`FEMAssembler::buildAffineSystem()` 是本工程把频域有限元 `curl-curl` 系统拆成 频率无关 部分的入口，是 ALPS 快速扫频 (`AlpsSweep`) 以及 Krylov 类模型降阶 (MOR) 方法可以直接消费的标准化输入。本文以 Nédélec/层次向量有限元、微波器件三维切向矢量有限元、TFE 与 ALPS/MOR 文献为背景 [1, 2, 3, 4, 5, 6, 7, 8]，从频域 Maxwell 方程和棱元 Galerkin 离散出发，推导该函数返回的矩阵、端口耦合向量和右端项的物理含义，并把推导各步骤对回到 `src/fem/FEMAssembler.cpp`、`src/bc/WavePortBC.cpp` 与 `src/sweep/AlpsSweep.cpp` 中的实现。

目录

1	为什么需要“仿射”系统	2
2	从 Maxwell 方程到 <code>curl-curl</code> 弱形式	2
2.1	Galerkin 棱元离散	3
3	$A(\omega)$ 的仿射拆分	3
3.1	无损情形：单一 M	3
3.2	有损情形：单一质量矩阵不再充分	4
4	端口边界项的秩 1 仿射拆分	4
4.1	端口模的来源：默认走二维端面本征	4
4.2	从端面本征到端口算子的代数推导	4
4.3	虚拟端口列表	6
5	对应到 <code>buildAffineSystem()</code> 的实现	6
5.1	第一步：单元参考矩阵 K^T, M^T 缓存	6
5.2	第二步：稀疏图 + 体积装配 (K, M)	7
5.3	第三步：损耗标志	7
5.4	有损介质未仿射化	7
5.5	第四步：虚拟端口展开	8
6	PEC 零 Dirichlet 投影	9
7	装配后的不变量	9
8	下游用法：构造 <code>AlpsSweep</code> 的 ROM 矩阵	10

1 为什么需要”仿射”系统	2
9 当前限制与扩展点	11
10 小结	11
11 下游用法：构造 AlpsSweep 的 ROM 矩阵	12
11.1 从全空间仿射系统到频点矩阵	12
11.2 离线阶段：以中心频率建立展开基	13
11.3 ROM 矩阵是如何投影出来的	14
11.4 为什么端口项在 ROM 中仍保持秩 1	14
11.5 和 ALPS 文献中的矩阵观点对应起来	14
11.6 为何 buildAffineSystem() 足够	15

1 为什么需要”仿射”系统

频域电磁有限元在每个频点 ω （角频率， $\omega = 2\pi f$ ）处都需要装配并求解一个大型复对称稀疏线性系统

$$\mathbf{A}(\omega) \mathbf{x}(\omega) = \mathbf{b}(\omega). \quad (1)$$

若每个频点都从零开始走完”算单元矩阵 $\rightarrow$ 累加全局矩阵 $\rightarrow$ 因式分解 $\rightarrow$ 回代”四步，扫频成本随频点数线性增长。把 $\mathbf{A}(\omega)$ 写成

$$\mathbf{A}(\omega) = \sum_q f_q(\omega) \mathbf{A}_q \quad (2)$$

的”仿射展开”（affine decomposition）后，所有 $\mathbf{A}_q$ 都是频率无关的稀疏矩阵，只在 ω 上做轻量加权。这正是 Krylov MOR、AWE/ALPS 等快速扫频方法赖以工作的代数前提。FEMAssembler::buildAffineSystem 就是一次性产出 (2) 中所有 $\mathbf{A}_q$ 及其加权函数所需要的物理量。

下文先在§2 推导 $\mathbf{A}(\omega)$ 的连续与离散形式，§3 给出仿射拆分公式，§4 推导端口算子的秩 1 拆分。§5 把这些数学量逐项映射到 C++ 实现，§7–9 讨论实现层的不变量、PEC 投影、损耗判定与目前的限制。

2 从 Maxwell 方程到 curl-curl 弱形式

约定时间因子 $e^{j\omega t}$ 。各向同性线性介质中，频域 Maxwell 方程

$$\nabla \times \mathbf{E} = -j\omega\mu\mathbf{H}, \quad (3)$$

$$\nabla \times \mathbf{H} = (\sigma + j\omega\varepsilon)\mathbf{E} + \mathbf{J}_s \quad (4)$$

消去 $\mathbf{H}$ 得电场 curl-curl 方程

$$\nabla \times (\mu_r^{-1} \nabla \times \mathbf{E}) - k_0^2 \varepsilon_c(\omega) \mathbf{E} = -j\omega\mu_0 \mathbf{J}_s, \quad (5)$$

其中

$$k_0 = \omega\sqrt{\mu_0\varepsilon_0}, \quad \varepsilon_c(\omega) = \varepsilon_r - j\frac{\sigma}{\omega\varepsilon_0}. \quad (6)$$

记计算域为 Ω ，端口面记作 $\Gamma_p \subset \partial\Omega$ ($p = 1, \dots, N_p$)，PEC 面记作 Γ_e 。乘以测试函数 $\mathbf{v} \in H_0(\text{curl}; \Omega)$ （在 Γ_e 上满足 $\mathbf{n} \times \mathbf{v} = 0$ ）并对 Ω 积分，对 curl 项做分部积分，得到弱形式

$$\int_{\Omega} \mu_r^{-1} (\nabla \times \mathbf{v}) \cdot (\nabla \times \mathbf{E}) dV - k_0^2 \int_{\Omega} \varepsilon_c \mathbf{v} \cdot \mathbf{E} dV + \sum_p \int_{\Gamma_p} \mathbf{v} \cdot (\mathbf{n} \times \mu_r^{-1} \nabla \times \mathbf{E}) dS = (\text{右端}). \quad (7)$$

最关键的一步是把 Γ_p 上的边界项代换为只用切向电场表达的端口算子，推导见§4。这里的边界项符号采用外法向 $\mathbf{n}$ 与时间因子 $e^{j\omega t}$ ；若端口模的传播方向或相量约定改变，端口算子的整体符号会随之改变。本工程的实现约定与 `WavePortBC::applyOneMode` 中的 `admittance = j beta` 一致。

2.1 Galerkin 棱元离散

把 $\mathbf{E}$ 在棱元 (Nédélec H(curl) 一阶不完备 / 二阶半完备) 基 $\{\mathbf{N}_i\}_{i=1}^N$ 上展开

$$\mathbf{E} \approx \sum_i x_i \mathbf{N}_i, \quad (8)$$

取 $\mathbf{v} = \mathbf{N}_a$, $\mathbf{E}$ 的展开系数为 x_b , 则弱形式 (7) 中的体积分两项直接给出全局矩阵

$$A_{ab}^{\text{vol}}(\omega) = \underbrace{\int_{\Omega} \mu_r^{-1} (\nabla \times \mathbf{N}_a) \cdot (\nabla \times \mathbf{N}_b) dV}_{\equiv K_{ab}} - k_0^2 \underbrace{\int_{\Omega} \varepsilon_c \mathbf{N}_a \cdot \mathbf{N}_b dV}_{\equiv \tilde{M}_{ab}(\omega)}. \quad (9)$$

注意 K_{ab} 完全频率无关, 而 $\tilde{M}_{ab}(\omega)$ 因含 $\varepsilon_c(\omega)$ 在 $\sigma \neq 0$ 时仍是频率函数。

按单元 T 分解, 标准元素装配

$$K_{ab} = \sum_T \frac{1}{\mu_{r,T}} \mathbf{K}_{ab}^T, \quad \mathbf{K}_{ab}^T = \int_T (\nabla \times \mathbf{N}_a) \cdot (\nabla \times \mathbf{N}_b) dV, \quad (10)$$

$$\tilde{M}_{ab}(\omega) = \sum_T \varepsilon_{c,T}(\omega) \mathbf{M}_{ab}^T, \quad \mathbf{M}_{ab}^T = \int_T \mathbf{N}_a \cdot \mathbf{N}_b dV. \quad (11)$$

$\mathbf{K}^T, \mathbf{M}^T$ 只依赖单元几何与基函数, 不依赖 ω , 是频率无关的单元参考矩阵。

3 $A(\omega)$ 的仿射拆分

3.1 无损情形：单一 M

无损情形 $\sigma \equiv 0$, 因此 $\varepsilon_c = \varepsilon_r$ 也与 ω 无关。体积部分立刻退化为

$$\mathbf{A}^{\text{vol}}(\omega) = \mathbf{K} - k_0^2(\omega) \mathbf{M}, \quad (12)$$

其中

$$\mathbf{K} = \sum_T \frac{1}{\mu_{r,T}} \mathbf{K}^T, \quad \mathbf{M} = \sum_T \varepsilon_{r,T} \mathbf{M}^T. \quad (13)$$

两者都是实对称稀疏矩阵, 与 ω 完全解耦。把端口算子 (§4) 合并进来后, 传播模对应的线性系统可写为

$$\boxed{\mathbf{A}(\omega) = \mathbf{K} - k_0^2 \mathbf{M} + \sum_{v=1}^{N_v} Y_v(\omega) \mathbf{m}_v \mathbf{m}_v^T}, \quad \boxed{\mathbf{b}(\omega) = 2 \sum_{v=1}^{N_v} Y_v(\omega) a_v^{\text{inc}} \mathbf{m}_v} \quad (14)$$

其中 N_v 是虚拟端口数 (每个物理端口的每个保留模式各一个), $\mathbf{m}_v$ 是端口耦合稀疏向量,

$$Y_v(\omega) = j \beta_v^+(\omega), \quad \beta_v^+(\omega) = \sqrt{\max(0, k_0^2 - k_{c,v}^2)} \quad (15)$$

是当前 affine/ALPS 路径实际使用的简化模导纳。 a_v^{inc} 为入射模幅度; 未被标为 `isExcitationMode` 的虚拟端口有 $a_v^{\text{inc}} = 0$ 。单频点 direct path 为了数值稳健, 在 $\beta_v^+ = 0$ 时使用 $j \cdot k_0$ 的占位导纳; 该低于截止模处理不是严格的波导模导纳, 也不是当前 ALPS reduced evaluation 的一部分, 见§9。

3.2 有损情形：单一质量矩阵不再充分

若存在 $\sigma > 0$, 则 (6) 给出 $\varepsilon_c(\omega) = \varepsilon_r - j\sigma/(\omega\varepsilon_0)$, (9) 中的质量项变为

$$\tilde{M}_{ab}(\omega) = \sum_T \left(\varepsilon_{r,T} - j \frac{\sigma_T}{\omega\varepsilon_0} \right) \mathbf{M}_{ab}^T = \varepsilon_r\text{-加权全局}M - \frac{j}{\omega\varepsilon_0} (\sigma\text{-加权全局}M). \quad (16)$$

这等价于把单一 M 拆成两个全局参考矩阵

$$\mathbf{M} = \sum_T \varepsilon_{r,T} \mathbf{M}^T, \quad \mathbf{M}_\sigma = \sum_T \sigma_T \mathbf{M}^T, \quad (17)$$

若显式缓存 M_σ , 体积项仍可写成多项仿射形式

$$\mathbf{A}(\omega) = \mathbf{K} - k_0^2 \mathbf{M} + \frac{jk_0^2}{\omega\varepsilon_0} \mathbf{M}_\sigma + \sum_v Y_v(\omega) \mathbf{m}_v \mathbf{m}_v^\top. \quad (18)$$

当前实现尚未缓存 M_σ , 所以遇到 $\sigma > 0$ 时 `AffineSystem::lossless` 被置为 `false`, 调用方 (`AlpsSweep`) 改走逐频点 `direct path`。具体判定见§5.3。

4 端口边界项的秩 1 仿射拆分

4.1 端口模的来源：默认走二维端面本征

仿射拆分需要每个端口面 Γ_p 上的“传播模式形状” $\mathbf{e}_{p,m}^\perp(\mathbf{r})$ 和对应截止波数 $k_{c,p,m}^2$ 。这种把端口区模函数与三维有限元未知量用变分形式耦合的思想，与微波器件 FEM/TFE 文献中的端口模展开一致 [4, 5]。本工程现支持三条路径产出这两个量，全部最终落到同一 `PortMode` 数据契约上：

1. **NPM** (`--port-method numerical`, 默认): 把端口面 Γ_p 上的三角剖分单独抽出来当作 2D 网，在其上做 $H(\text{curl})$ 棱元离散，求解二维广义本征值问题

$$\mathbf{K}_{\text{port}} \mathbf{v} = k_c^2 \mathbf{M}_{\text{port}} \mathbf{v}, \quad (19)$$

取最低非伪本征对 $(\mathbf{v}_1, k_{c,1}^2)$ 作为主模。模式形状由 $\mathbf{e}^\perp = \sum_i v_i \mathbf{N}_i^\partial$ 给出，其中 $\mathbf{N}_i^\partial$ 是端口面切向 $H(\text{curl})$ 棱元基。对任何二维截面（矩形 / 脊形 / 圆形 / 偏心）都成立。实现：`PortModeSolver::solve(faceId) → computeMode → computeMultiMode(faceId, 1)`。

2. **APM** (`--port-method analytic`): 直接套矩形波导 TE10 闭式 $\mathbf{e}_{\text{TE10}} = \sqrt{2/(ab)} \sin(\pi u/a) \hat{v}$, $k_c^2 = (\pi/a)^2$, 再 L2 投影到 FE 端口 dof。仅矩形截面可用。
3. **TFE** (`--port-method tfe`): 与 NPM 同一本征解 (19), 但保留最低 N 个非伪本征对，每个模出一个独立的虚拟端口。

NPM 和 TFE 共用 `PortModeSolver::computeMultiMode`: 单元矩阵对应面 $H(\text{curl})$ 基的 curl-curl 与 mass , 三角形 7 点 Wandzura 求积；本征求解走 `LAPACKE_dsygv` (无 MKL 时回落 Jacobi 迭代)；零本征对应离散梯度零空间，按相对阈值剔除。当前本征式 (19) 未显式带入端口截面材料张量，因此等价于空气/均匀归一化端口；若后续支持介质填充或色散端口，应把 β^2 推广为 $k_0^2 \mu_r \varepsilon_r - k_c^2$ 或直接构造带材料权重的端面本征问题。

4.2 从端面本征到端口算子的代数推导

设 Γ_p 上传播波导支持 $N_{m,p}$ 个传播或近截止模 $\mathbf{e}_{p,m}^\perp(\mathbf{r})$, $m = 1, \dots, N_{m,p}$ 。无论这些模是来自数值本征 (19) 还是闭式 TE10, 当前 `affine` 路径只把传播部分写成

$$\beta_{p,m}^+(\omega) = \sqrt{\max(0, k_0^2 - k_{c,p,m}^2)}, \quad Y_{p,m}(\omega) = j\beta_{p,m}^+(\omega). \quad (20)$$

在 Γ_p 上令真正的切向投影为

$$\mathbf{E}_t = (\mathbf{I} - \mathbf{n}\mathbf{n}^\top)\mathbf{E} = -\mathbf{n} \times (\mathbf{n} \times \mathbf{E}). \quad (21)$$

再把切向电场展开为

$$\mathbf{E}_t|_{\Gamma_p} = \sum_{m=1}^{N_{m,p}} \alpha_{p,m} \mathbf{e}_{p,m}^\perp + \mathbf{E}^{\text{inc}}, \quad (22)$$

代入 (7) 中的端口边界积分后，无源端口在弱形式中贡献的部分是

$$\int_{\Gamma_p} \mathbf{v} \cdot \left[\sum_m Y_{p,m}(\omega) (\mathbf{e}_{p,m}^\perp \cdot \mathbf{E}_t) \mathbf{e}_{p,m}^\perp \right] dS, \quad (23)$$

这一步的来源是把端口边界上的切向场先展开，再用模正交性把边界算子投影回单个模方向。更具体地说，若将

$$\mathbf{E}_t|_{\Gamma_p} = \sum_m \alpha_{p,m} \mathbf{e}_{p,m}^\perp + \mathbf{E}^{\text{inc}} \quad (24)$$

代入端口边界条件，那么对第 m 个传播模会得到一个标量关系

$$\mathbf{n} \times \mu_r^{-1} \nabla \times \mathbf{E} = - \sum_m Y_{p,m}(\omega) \alpha_{p,m} \mathbf{e}_{p,m}^\perp + 2 \sum_m Y_{p,m}(\omega) a_{p,m}^{\text{inc}} \mathbf{e}_{p,m}^\perp. \quad (25)$$

这里第一项对应“由未知场自身反射回去的反应电流”，第二项对应“已知入射波注入的激励电流”。把它代回 (7) 的边界积分后，得到双线性项

$$\sum_m Y_{p,m}(\omega) \int_{\Gamma_p} (\mathbf{v} \cdot \mathbf{e}_{p,m}^\perp) (\mathbf{E}_t \cdot \mathbf{e}_{p,m}^\perp) dS, \quad (26)$$

以及对应的右端项

$$\mathbf{f}_p^{\text{port}}(\omega) = 2 \sum_m Y_{p,m}(\omega) a_{p,m}^{\text{inc}} \int_{\Gamma_p} (\mathbf{v} \cdot \mathbf{e}_{p,m}^\perp) dS. \quad (27)$$

对有限元试函数取 $\mathbf{v} = \mathbf{N}_a$ 后，定义端口耦合向量

$$[\mathbf{m}_{p,m}]_a = \int_{\Gamma_p} \mathbf{N}_a \cdot \mathbf{e}_{p,m}^\perp dS, \quad (28)$$

就得到矩阵形式的秩 1 算子

$$\sum_m Y_{p,m}(\omega) \mathbf{m}_{p,m} \mathbf{m}_{p,m}^\top, \quad (29)$$

以及右端

$$\mathbf{f}_p^{\text{port}}(\omega) = 2 \sum_m Y_{p,m}(\omega) a_{p,m}^{\text{inc}} \mathbf{m}_{p,m}. \quad (30)$$

因此，公式右端中的系数“2”并不是经验因子，而是端口边界条件里入射波与反射波叠加后留下的标准结果：当把总场写成“未知散射场 + 已知入射场”时，未知场产生一个负号，入射场又额外贡献一次同样大小的项，二者合并后就得到 2。

从散射参数的角度看，这个右端项也可以这样理解：端口模展开通常把每个虚拟端口上的场写成“入射幅度 $a_{p,m}^{\text{inc}}$ + 反射幅度 $a_{p,m}^{\text{ref}}$ ”，而目标求解的并不是单独的边界场，而是给定入射激励后系统对散射波的响应。对于传播模，边界条件实际上把未知的反射波通量与端口导纳 $Y_{p,m}(\omega)$ 绑定起来，因此当把边界算子移到弱式右端时，已知的入射幅度就会以线性源项的形式出现，并且与同一个耦合向量 $\mathbf{m}_{p,m}$ 相乘。这就是为什么端口散射参数、模导纳和有限元右端

项在代数上最终会写成同一个低秩结构：前者定义物理输入输出，后者则是其有限元离散后的矩阵表达。

$\mathbf{m}_{p,m}$ 在代码中称为 `couplingWeights`，是稀疏的——只有支撑落在 Γ_p 的边自由度的分量非零。 $\mathbf{m}_{p,m}$ 与 ω 无关； $Y_{p,m}(\omega)$ 是端口项的频率函数。若该模式带入射幅度 $a_{p,m}^{\text{inc}}$ ，右端同步获得 $2Y_{p,m}a_{p,m}^{\text{inc}}\mathbf{m}_{p,m}$ ，这正对应 `WavePortBC::applyOneMode` 中的 `rhs += 2.0 * admittance * incident * rowValue`。NPM/TFE 路径下 $\mathbf{m}_{p,m}$ 直接由 $\mathbf{M}_{\text{port}}\mathbf{v}_m$ 给出（ M -内积投影），APM 路径下 $\mathbf{m}_{p,m}$ 是闭式 $\mathbf{e}^\perp$ 在 $H(\text{curl})$ 端口 dof 上的 L2 投影。三者共用同一 `PortMode::couplingWeights` 字段。`computeMultiMode` 会把本征向量按 $\mathbf{v}_m^T \mathbf{M}_{\text{port}} \mathbf{v}_m = 1$ 归一化，再保存 $\mathbf{m}_m = \mathbf{M}_{\text{port}} \mathbf{v}_m$ ；因此 $\mathbf{m}_m^T \mathbf{x}$ 就是离散场在该端口模上的 M -内积投影，而不是原始边 dof 系数。

4.3 虚拟端口列表

把 (29) 对所有物理端口、所有解析模式平铺成一张虚拟端口 列表，(14) 中的端口和写成

$$\sum_{v=1}^{N_v} Y_v(\omega) \mathbf{m}_v \mathbf{m}_v^T, \quad N_v = \sum_p N_{m,p}, \quad (31)$$

其中索引 v 与 `(projectPortIndex, modeIndex)` 一一对应。单模 NPM/APM 路径下 $N_{m,p} = 1$ ， $N_v = N_p$ 直接退化。TFE 多模情况下，每个物理端口可能贡献若干虚拟端口；最多一个被标记为 `isExcitationMode`，承载入射幅相，其余作纯吸收端口（在 (7) 中入射项 $\mathbf{E}^{\text{inc}}$ 为零）。

5 对应到 `buildAffineSystem()` 的实现

接口契约见 `include/bpfem/fem/FEMAssembler.hpp` 的 `AffineSystem`:

```
struct AffineSystem {
    SparseMatrix K; // sum_e (1/mu_r,e) * K_e
    SparseMatrix M; // sum_e (eps_r,e) * M_e
    std::vector<std::vector<std::pair<int,double>>> portCoupling; // m_p (per virtual port)
    std::vector<double> portCutoffSquared; // k_{fc,p}^2
    std::vector<int> portFaceIds; // physical face id
    std::vector<int> projectPortIndex; // index into project_.ports
    std::vector<bool> isExcitationMode; // true = incident-bearing mode
    std::unordered_set<int> constrainedDofs; // PEC zero-Dirichlet mask
    bool lossless = true; // false if any sigma > 0
};
```

下面分四步把数学量映射到代码（行号引自 `src/fem/FEMAssembler.cpp`）。

5.1 第一步：单元参考矩阵 $\mathbf{K}^T, \mathbf{M}^T$ 缓存

`ensureElementCache()` 对每个有效四面体 T 计算一次几何 + 基函数组合

$$(\mathbf{K}_{ab}^T, \mathbf{M}_{ab}^T) = \left(\sum_q w_q V_T (\nabla \times \mathbf{N}_a) \cdot (\nabla \times \mathbf{N}_b), \sum_q w_q V_T \mathbf{N}_a \cdot \mathbf{N}_b \right) \quad (32)$$

其中 $\{(w_q, \lambda_q)\}$ 是 `tetraQuadraturePoints()` 给出的 11 点四面体重心坐标求积公式。对应代码：

```
for (a = 0..dof) for (b = a..dof)
    for each quadrature point qp:
```

```

    curlCurl += qp.weight * volume * dot(rowBasis.curl, colBasis.curl);
    mass += qp.weight * volume * dot(rowBasis.value, colBasis.value);
entry.curlCurlGeo[idx] = curlCurl; //  $K^T_{ab}$ 
entry.massGeo [idx] = mass; //  $M^T_{ab}$ 
entry.invMu = 1 / mu_r,T;
entry.epsR = eps_r,T;
entry.conductivity = sigma_T; // used for lossless check

```

K^T, M^T 用上三角紧致索引存储 upperTriIndex(a,b,dof), 保证装配后 K, M 都是 CSR 上三角对称矩阵。

5.2 第二步：稀疏图 + 体积装配 (K, M)

buildAffineSystem() 不调用与 assemble() 共享的稀疏图，而是单独构造一个只含体积耦合的稀疏图：

```

const auto pattern = volumePatternFor(out.constrainedDofs);
out.K = assembleVolumePiece(pattern, out.constrainedDofs, /*useCurlCurl=*/true);
out.M = assembleVolumePiece(pattern, out.constrainedDofs, /*useCurlCurl=*/false);

```

理由：ALPS 离线保存的是体积矩阵 K, M 加若干低秩端口向量 m_v ，而不是把 $m_v m_v^T$ 预先写进稀疏矩阵。因此 K, M 的稀疏图 严格小于等于 单频点 direct path 的图；端口外积在 ROM 投影或 reduced evaluation 中按低秩形式单独处理。assembleVolumePiece 内对每个单元做

$$K += \frac{1}{\mu_{r,T}} K^T, \quad M += \varepsilon_{r,T} M^T, \quad (33)$$

其中 K^T, M^T 来自第一步缓存， σ_T 项故意不进入 M ，保留实矩阵性质（满足 (13)）。

5.3 第三步：损耗标志

```

out.lossless = true;
for (const auto& entry : elementCache_) {
    if (entry.valid && entry.conductivity > 0.0) {
        out.lossless = false;
        break;
    }
}

```

对应数学事实：单一实对称 M 不再能表达 $\varepsilon_c(\omega)$ 在 $\sigma > 0$ 时的 j/ω 频率依赖，因此 (14) 不再充分。AlpsSweep 读到 lossless==false 时直接抛 runtime_error 提示 --sweep direct。

5.4 有损介质未仿射化

当计算域中存在 $\sigma_T > 0$ 的单元时，buildAffineSystem() 目前不会继续尝试把有损项写成完整的仿射和，而是直接把 out.lossless 置为 false。从连续方程看，问题出在复介电常数

$$\varepsilon_c(\omega) = \varepsilon_r - j \frac{\sigma}{\omega \varepsilon_0} \quad (34)$$

所引入的频率依赖并非单一的 $k_0^2(\omega)$ 权重，而是同时包含一个 ω^{-1} 因子。于是体积项在弱式中可分解为

$$-k_0^2(\omega) \int_{\Omega} \varepsilon_r \mathbf{v} \cdot \mathbf{E} dV + j \frac{k_0^2(\omega)}{\omega \varepsilon_0} \int_{\Omega} \sigma \mathbf{v} \cdot \mathbf{E} dV. \quad (35)$$

若希望继续保持“离线缓存固定矩阵、在线仅做标量加权”的仿射框架，则除 $\mathbf{K}$ 与 $\mathbf{M}$ 外，还必须额外引入导电质量矩阵

$$\mathbf{M}_\sigma = \sum_T \sigma_T \mathbf{M}^T, \quad (36)$$

才能把系统写成

$$\mathbf{A}(\omega) = \mathbf{K} - k_0^2(\omega) \mathbf{M} + j \frac{k_0^2(\omega)}{\omega \varepsilon_0} \mathbf{M}_\sigma + \sum_v Y_v(\omega) \mathbf{m}_v \mathbf{m}_v^T. \quad (37)$$

但这还只是“形式上可写”，并不能直接等价于当前 ALPS 实现可消费的标准仿射结构，因为在线递推和 ROM 评估会同时遇到两个问题：

1. $\mathbf{M}_\sigma$ 带来的系数是 $j k_0^2(\omega)/(\omega \varepsilon_0)$ ，它不是简单的多项式函数，也没有被当前 Krylov 递推显式匹配；
2. 端口项与体积有损项耦合后，若要保持严格的矩匹配或有理近似，还需要对展开点附近的频率导数进行额外处理。

因此，当前实现选择更保守的策略：只要存在任何 $\sigma_T > 0$ ，就不再给出“可直接用于 ROM 的 lossless affine system”，而是把全空间求解交给 `assembler.assemble(omega)` 的逐频点 direct path。

从工程实现角度，这个判定等价于：离线阶段只缓存 $\mathbf{K}$ 和 $\mathbf{M}$ 这两块频率无关矩阵；当有损介质出现时，若不额外实现 $\mathbf{M}_\sigma$ 的缓存、在线重组和相应的 ROM 递推更新，继续使用原先的 `AlpsSweep` 低维基会导致幅相误差在频带内累积，尤其是在导电损耗较强、共振峰较尖锐时更明显。故而 `lossless==false` 并不是“无法求解”，而是“当前这套仿射 ROM 路径未覆盖该物理情形”的显式标记。

5.5 第四步：虚拟端口展开

最后由注册的 `IBoundaryCondition` 列表（典型为 `WavePortBC`）通过 `affineContributions(ctx)` 给出每个虚拟端口的 $(\mathbf{m}_v, k_{c,v}^2, \dots)$ 。`WavePortBC::affineContributions` 内部对每个 `ProjectPort` 先看 `portModeSolver.multiMode(faceId)`：

1. TFE 多模 (`multiMode != nullptr`)：按 `modeIndex` 顺序把每个解析模 m 都打成一个 `AffinePortContribution`，`isExcitationMode` 仅在 $m = \text{excitationModeIndex}$ 且 `port.excited` 同时成立时为真；
2. NPM/APM 单模 (`multiMode == nullptr`)：直接调用 `portModeSolver.solve(faceId)`（NPM 走二维 $\mathbf{H}(\text{curl})$ 本征，APM 走闭式 TE10 投影），产出唯一一个虚拟端口。

对每个 `contribution` 内部还会过滤掉落在 `ctx.constrainedDofs` 里的 `dof`，保证 $\mathbf{m}_v$ 与 $\mathbf{K}, \mathbf{M}$ 共用同一全局 `dof` 编号空间。汇总循环是：

```
for (const auto& cond : bcs_) {
    if (!cond) continue;
    for (auto& contrib : cond->affineContributions(ctx)) {
        out.portFaceIds.push_back (contrib.faceId);
        out.portCutoffSquared.push_back(contrib.cutoffSquared);
        out.portCoupling.push_back (std::move(contrib.coupling));
        out.projectPortIndex.push_back(contrib.projectPortIndex);
        out.isExcitationMode.push_back(contrib.isExcitationMode);
    }
}
```


把这四步串起来, `buildAffineSystem()` 实际给出的就是 (14) 所需要的矩阵与端口素材; 右端 $\mathbf{b}(\omega)$ 不直接缓存, 而是由 `projectPortIndex`、`isExcitationMode`、`ProjectPort::magnitudeW/phaseDeg` 以及 `powerNormalizationFactor` 在 `AlpsSweep::evaluate` 中重建:

数学量	代码字段	装配位置
$\mathbf{K} = \sum_T \mu_{r,T}^{-1} \mathbf{K}^T$	<code>out.K</code>	§5.2 <code>assembleVolumePiece(...,true)</code>
$\mathbf{M} = \sum_T \varepsilon_{r,T} \mathbf{M}^T$	<code>out.M</code>	§5.2 <code>assembleVolumePiece(...,false)</code>
$\mathbf{m}_v$ (每虚拟端口)	<code>out.portCoupling[v]</code>	§5.5 <code>affineContributions</code>
$k_{c,v}^2$	<code>out.portCutoffSquared[v]</code>	同上
$v \rightarrow$ 物理端口面 id	<code>out.portFaceIds[v]</code>	同上
$v \rightarrow$ ProjectPort 索引	<code>out.projectPortIndex[v]</code>	同上
是否激励模	<code>out.isExcitationMode[v]</code>	同上
入射幅相 a_v^{inc}	<code>project.ports[...]</code>	<code>AlpsSweep::evaluate</code>
PEC 约束 dof 集	<code>out.constrainedDofs</code>	§6 <code>cachedConstrainedDofs</code>
$\sigma \equiv 0$ 判定	<code>out.lossless</code>	§5.3

表 1: `AffineSystem` 字段与数学模型量的一一对应。

6 PEC 零 Dirichlet 投影

PEC 面 Γ_e 上要求 $\mathbf{n} \times \mathbf{E} = 0$, 棱元离散后即“ Γ_e 上所有切向边 dof 强置零”。代码用 `collectPecConstrainedDofs()` 扫一次表面三角列表, 把不属于任何端口的边全部塞进 `cachedConstrainedDofs`。这个集合在 `buildAffineSystem()` 里有三处用途:

1. **稀疏图**: `volumePatternFor` 中跳过任意端含约束 dof 的物理耦合 (a,b) 对, 但为每个约束 dof 保留一个对角占位, 保证矩阵维度仍等于全局边 dof 数。
2. **累加**: `assembleVolumePiece` 中重复同样的过滤, 单元贡献不写入约束行/列。
3. **端口耦合**: `WavePortBC::affineContributions` 中对每个 $\mathbf{m}_v$ 内部过滤约束 dof, 避免端口耦合向量的支撑超出 $\mathbf{K}, \mathbf{M}$ 的稀疏图。

注意 `buildAffineSystem()` 不在 $\mathbf{K}, \mathbf{M}$ 上做 `imposeZeroDirichlet` 那样的“主对角线置 1”操作; 它选择的是投影 路径——所有涉及约束 dof 的物理贡献均为零, 只保留对角占位以维持全局维度。数学上这等价于在子空间 $\{x_i = 0, i \in \text{PEC}\}$ 上写下系统; 约束 dof 集仍随 `AffineSystem` 传给 `AlpsSweep`, 方便下游做一致性检查或重建满空间向量。离线阶段用于生成 Krylov 基的 $\mathbf{A}_0$ 仍来自 `assembler.assemble(...)`, 其中 `direct path` 会对这些对角占位施加单位对角。

7 装配后的不变量

1. **$\mathbf{K}, \mathbf{M}$ 实对称**: 上三角 CSR 复矩阵但虚部恒为 0, 由 (13) 本身的实数性保证; 引入张量介质后需要扩展为多分量缓存。
2. **虚拟端口排序**: `out.portCoupling` 等数组按 “`projectPortIndex` 升序、其内 `modeIndex` 升序” 排列, 与 `WavePortBC::affineContributions` 的循环次序一致。 `AlpsSweep` 中的 `dominantVirtualByProject` 依赖这个顺序定位每个物理端口的主模虚拟端口。

3. **激励唯一性**：每个 `projectPortIndex` 至多一个 `isExcitationMode==true`。NPM/APM 单模情形下”唯一”等价于”全部”。
4. **$\mathbf{m}_v$ 与 $\mathbf{K}, \mathbf{M}$ 同 dof 空间**：约束 dof 在三处一致剔除，所以 $\mathbf{m}_v$ 不会引用被投影掉的未知量；但 $\mathbf{m}_v \mathbf{m}_v^\top$ 一般会在端口 dof 间形成比体积矩阵更稠密的外积，因此不能要求它与 $\mathbf{K}, \mathbf{M}$ 同稀疏图。当前 affine path 正是把这些端口项保存为低秩向量而非稀疏矩阵。
5. **复对称投影**：下游 ALPS 用不带共轭的双线性 $\mathbf{V}^\top \mathbf{K} \mathbf{V}$ 、 $\mathbf{V}^\top \mathbf{M} \mathbf{V}$ 与 $(\mathbf{V}^\top \mathbf{m}_v)(\mathbf{V}^\top \mathbf{m}_v)^\top$ 做 Galerkin 投影。 $\mathbf{K}, \mathbf{M}$ 的实对称性 + 端口低秩外积的复对称性共同保证 reduced matrix 在 (14) 下保留复对称结构。

8 下游用法：构造 AlpsSweep 的 ROM 矩阵

为完整说明 `buildAffineSystem()` 输出的”够用性”，简述下游 `AlpsSweep::buildOffline` 对它的消费方式。与其把这一过程描述成普通代码流程，不如把它写成更接近论文中的算法环境，以便突出“离线构基—在线评估”的两阶段结构。

Algorithm 1 由仿射系统构造 AlpsSweep 的 ROM

Require: 仿射数据 $\mathcal{A}_{\text{aff}} = (\mathbf{K}, \mathbf{M}, \{\mathbf{m}_v\}, \{k_{c,v}^2\}, \{a_v^{\text{inc}}\})$, 展开频率 ω_0 , 阶数 r

Ensure: ROM 基 $\mathbf{V}$ 及小矩阵 $\mathbf{K}_r, \mathbf{M}_r, \{\mathbf{m}_{v,r}\}$

```

1: 组装并因式分解  $\mathbf{A}_0 \leftarrow \mathbf{A}(\omega_0)$ 
2: for each virtual port  $v$  do
3:   Solve  $\mathbf{A}_0 \mathbf{w}_v^{(1)} = \mathbf{m}_v$  ▷ shift-and-invert 初始向量
4:   for  $k = 1$  to  $r - 1$  do
5:     Solve  $\mathbf{A}_0 \mathbf{w}_v^{(k+1)} = \mathbf{M} \mathbf{w}_v^{(k)}$ 
6:     Orthonormalize  $\mathbf{w}_v^{(k+1)}$  against existing basis vectors
7:   end for
8: end for
9: Assemble orthonormal basis  $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_r]$ 
10: Project  $\mathbf{K}_r \leftarrow \mathbf{V}^\top \mathbf{K} \mathbf{V}$ ,  $\mathbf{M}_r \leftarrow \mathbf{V}^\top \mathbf{M} \mathbf{V}$ 
11: for each virtual port  $v$  do
12:   Project  $\mathbf{m}_{v,r} \leftarrow \mathbf{V}^\top \mathbf{m}_v$ 
13: end for
14: return  $\mathbf{V}, \mathbf{K}_r, \mathbf{M}_r, \{\mathbf{m}_{v,r}\}$ 

```

在中心频率 ω_0 处，AlpsSweep 先调用 `assembler.assemble(omega0)` 得到与 direct path 完全一致的全空间矩阵 $\mathbf{A}_0$ 并做因式分解。在理想传播模 affine 记号下，它对应

$$\mathbf{A}_0 \approx \mathbf{K} - k_0^2(\omega_0) \mathbf{M} + \sum_v Y_v(\omega_0) \mathbf{m}_v \mathbf{m}_v^\top. \quad (38)$$

这里写成“约等于”是因为 direct path 在低于截止模上有 $\mathbf{j} \star \mathbf{k}_0$ 占位导纳，而当前 reduced evaluation 只重建 $Y_v = \mathbf{j} \beta_v^+$ 的传播部分。接着对每个虚拟端口求解 $\mathbf{A}_0 \mathbf{w}_v^{(1)} = \mathbf{m}_v$ 作为 shift-and-invert Krylov 子空间起始向量；递推

$$\mathbf{A}_0 \mathbf{w}_v^{(k+1)} = \mathbf{M} \mathbf{w}_v^{(k)} \quad (39)$$

(注意右端只用到频率无关的 $\mathbf{M}$) 生成 r 阶子空间，再做 modified Gram-Schmidt 正交化得到 $\mathbf{V}$ 。因此当前实现更准确地说是单点、 $\mathbf{M}$ -dominant 的 *shift-and-invert Krylov-Galerkin*

ROM: 它复用了 ALPS/矩匹配文献中“一次或少数几次全空间因式分解 + 小型有理模型”的思想 [6, 7, 8], 但没有显式构造 Lanczos 三对角矩阵, 也没有对 $\beta_v(\omega)$ 的导数项做严格 Padé 矩匹配。投影后的 ROM

$$\mathbf{K}_r = \mathbf{V}^\top \mathbf{K} \mathbf{V}, \quad \mathbf{M}_r = \mathbf{V}^\top \mathbf{M} \mathbf{V}, \quad \mathbf{m}_{v,r} = \mathbf{V}^\top \mathbf{m}_v \quad (40)$$

是几十到几百维的稠密小矩阵, 最终在每个频点 ω 求解

$$\left(\mathbf{K}_r - k_0^2 \mathbf{M}_r + \sum_v Y_v(\omega) \mathbf{m}_{v,r} \mathbf{m}_{v,r}^\top \right) \mathbf{x}_r(\omega) = \mathbf{b}_r(\omega) \quad (41)$$

其中 $\mathbf{b}_r(\omega) = 2 \sum_v Y_v(\omega) a_v^{\text{inc}} \mathbf{m}_{v,r}$, 仅激励虚拟端口贡献非零项。即可获得端口响应, 避免重建全空间场。整条链路对 $\mathbf{K}, \mathbf{M}, \mathbf{m}_v, k_{c,v}^2$ 的需求都由 `buildAffineSystem()` 一次性满足。

9 当前限制与扩展点

1. **有损介质未仿射化**: $\sigma > 0$ 时 `lossless==false`, 调用方走 `direct path`。延伸方向是缓存 $\mathbf{M}_\sigma = \sum_T \sigma_T \mathbf{M}^T$ 作为第三块全局矩阵, 仿射展开多一项 $j(k_0^2/(\omega\epsilon_0))\mathbf{M}_\sigma$ 。若要做严格矩匹配, Krylov 递推还需要把该项对展开变量的导数纳入; 当前实现并未这样做。
2. **各向异性介质**: (10)–(11) 假设 μ_r, ϵ_r 标量, 对张量介质需要把 $\mathbf{K}^T, \mathbf{M}^T$ 拆成多分量并各自加权。
3. **端口模式频率无关性**: (29) 假设 $\mathbf{e}_{p,m}^\perp$ 与 ω 无关, 这是 ALPS 仿射展开的代数前提。NPM/TFE 路径下这要求二维 $\mathbf{H}(\text{curl})$ 本征问题 (19) 的特征对 (在频带内不发生模式交叉的前提下) 频率无关, 对空气或常规均匀填充波导近似成立; 色散填充、强损耗端口或开放波导需要单独处理。APM 路径假设矩形 TE10 闭式同样频率无关。
4. **低于截止模**: `direct path` 当前用 $j*k_0$ 作为 $\beta^+ = 0$ 时的占位导纳, 而 ALPS `reduced evaluation` 只在 $\beta^+ > 0$ 时加入端口低秩项和入射右端。若 TFE 保留了频带内低于截止的高阶模, `direct/ALPS` 两条路径会在该部分端口算子上不完全一致; 严格模型应使用完整的复传播常数或分传播/衰减两类模分别仿射化。
5. **ABC/SIBC 边界条件**: 当前 `IBoundaryCondition::affineContributions` 默认空。吸收边界、阻抗边界一般给出多项形式 (含 $\sqrt{j\omega}$ 等), 若要纳入仿射展开需要额外的项。

10 小结

`FEMAssembler::buildAffineSystem()` 把频域 curl-curl FEM 系统在 `lossless` 假设下彻底拆成 (14) 的“频率无关矩阵 + 频率函数加权”形式, 为 ALPS/Krylov 类快速扫频提供可复用的代数基础。其实现遵循三条主线: 单元参考矩阵缓存 (§5.1)、体积全局矩阵装配 (§5.2)、经 `IBoundaryCondition` 分发的虚拟端口展开 (§5.5); PEC 投影 (§6) 和损耗标志 (§5.3) 作为两条横切机制保证后续 ROM 与单频点 `direct path` 的等价性。

参考文献

- [1] J. C. Nédélec, “Mixed finite elements in $\mathbb{R}^3$,” *Numerische Mathematik*, vol. 35, pp. 315–341, 1980.
- [2] R. D. Graglia, D. R. Wilton, and A. F. Peterson, “Higher order interpolatory vector bases for computational electromagnetics,” *IEEE Transactions on Antennas and Propagation*, vol. 45, no. 3, pp. 329–342, 1997.

- [3] J. P. Webb, “Hierarchal vector basis functions of arbitrary order for triangular and tetrahedral finite elements,” *IEEE Transactions on Antennas and Propagation*, vol. 47, no. 8, pp. 1244–1253, 1999.
- [4] Z. J. Cendes and J.-F. Lee, “The transfinite element method for modeling MMIC devices,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 36, no. 12, pp. 1639–1649, 1988.
- [5] J.-F. Lee, “Analysis of passive microwave devices by using three-dimensional tangential vector finite elements,” *International Journal of Numerical Modelling*, vol. 3, no. 4, pp. 235–246, 1990.
- [6] J. E. Bracken, D.-K. Sun, and Z. J. Cendes, “S-domain methods for simultaneous time and frequency characterization of electromagnetic devices,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 46, no. 9, pp. 1277–1290, 1998.
- [7] D.-K. Sun, Z. Cendes, and J.-F. Lee, “ALPS: A new fast frequency-sweep procedure for microwave devices,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 49, no. 2, pp. 398–402, 2001.
- [8] B. Anderson, J. E. Bracken, J. B. Manges, G. Peng, and Z. Cendes, “Full-wave analysis in SPICE via model-order reduction,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 52, no. 9, pp. 2314–2320, 2004.

11 下游用法：构造 AlpsSweep 的 ROM 矩阵

这一节专门回答一个问题：`buildAffineSystem()` 产出的 $\mathbf{K}, \mathbf{M}, \mathbf{m}_v, k_{c,v}^2$ 为什么足以在 `AlpsSweep` 中构造 ROM 矩阵。结合 ALPS 原始论文中“只做一次全空间分解、在线反复评估低维模型”的思路 [7, 6, 8]，这里可以把整个过程分成三层：(1) 频率无关的离线数据如何定义；(2) 这些数据如何通过 Krylov / Galerkin 投影生成小矩阵；(3) 小矩阵如何重建每个频点的端口响应。

11.1 从全空间仿射系统到频点矩阵

设 `buildAffineSystem()` 给出离线对象

$$\mathcal{A}_{\text{aff}} = (\mathbf{K}, \mathbf{M}, \{\mathbf{m}_v\}_{v=1}^{N_v}, \{k_{c,v}^2\}_{v=1}^{N_v}, \{a_v^{\text{inc}}\}_{v=1}^{N_v}). \quad (42)$$

在 lossless 且端口模频率独立的前提下，任意频点的全空间矩阵都可写成

$$\mathbf{A}(\omega) = \mathbf{K} - k_0^2(\omega)\mathbf{M} + \sum_{v=1}^{N_v} Y_v(\omega) \mathbf{m}_v \mathbf{m}_v^T, \quad Y_v(\omega) = j\beta_v^+(\omega). \quad (43)$$

如果存在入射激励，则右端满足

$$\mathbf{b}(\omega) = 2 \sum_{v \in \mathcal{E}} Y_v(\omega) a_v^{\text{inc}} \mathbf{m}_v, \quad (44)$$

其中 $\mathcal{E}$ 是被标记为激励模的虚拟端口集合。式 (43) 的关键意义是：频率依赖只出现在标量函数 $k_0^2(\omega)$ 与 $Y_v(\omega)$ 中，矩阵结构本身是固定的。这正是 `AlpsSweep` 能把“全矩阵扫频”转化成“少量小矩阵扫频”的根本原因。

11.2 离线阶段：以中心频率建立展开基

`AlpsSweep::buildOffline()` 首先选一个展开中心频率 ω_0 ，并调用 `assembler.assemble(omega0)` 得到与 direct path 一致的全空间矩阵

$$\mathbf{A}_0 := \mathbf{A}(\omega_0). \quad (45)$$

在理想传播模记号下， $\mathbf{A}_0$ 既包含体积项 $\mathbf{K} - k_0^2(\omega_0)\mathbf{M}$ ，也包含中心频率处的端口秩 1 项。随后对每个虚拟端口 v 解线性方程

$$\mathbf{A}_0 \mathbf{w}_v^{(1)} = \mathbf{m}_v, \quad (46)$$

这一步是 ALPS / AWE / Krylov 族方法中典型的 shift-and-invert 起步：把“端口耦合向量”经过 $\mathbf{A}_0^{-1}$ 滤波后，得到最能反映该端口局部频响的初始方向。如果某个端口有多个保留模，则每个虚拟端口各自产生一个起始向量。

接下来，`AlpsSweep` 不再直接使用原始的频域矩阵，而是用固定的递推关系扩展子空间。当前实现的核心递推是

$$\mathbf{A}_0 \mathbf{w}_v^{(k+1)} = \mathbf{M} \mathbf{w}_v^{(k)}, \quad k = 1, 2, \dots, r-1. \quad (47)$$

这里右端只用到频率无关的 $\mathbf{M}$ ，因此每次递推都等价于“先乘一次质量矩阵，再用同一个分解好的 $\mathbf{A}_0$ 回代”。这就把大量频点的复杂性压缩进了一个公共子空间中。

为了避免不同端口、不同阶次的向量线性相关，递推得到的所有向量会做 modified Gram-Schmidt 正交化，形成正交基矩阵

$$\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_r], \quad \mathbf{V}^T \mathbf{V} = \mathbf{I}. \quad (48)$$

这一步在实现上可以概括为：对每个虚拟端口分别启动一次 shift-and-invert 递推，逐阶生成向量，再把全体向量汇总后做正交化与去重。其算法结构可写成如下伪代码。

Listing 1: AlpsSweep 离线阶段的虚拟端口子空间构造伪代码

```

Input:
  A0 = assemble(omega0) // center-frequency full matrix
  M // frequency-independent mass matrix
  virtualPorts = {m_v}
  r // target subspace order per start vector
Output:
  V // orthonormal basis for ROM

W = []
for each virtual port v in virtualPorts:
  w1 = solve(A0, m_v)
  append w1 to W

  w_prev = w1
  for k = 1 .. r-1:
    rhs = M * w_prev
    w_next = solve(A0, rhs)
    append w_next to W
    w_prev = w_next

V = []
for each w in W:
  for each q in V:

```

```

    w = w - q * (q^T * w) // modified Gram-Schmidt, 1st pass
    for each q in V:
        correction = q * (q^T * w) // optional reorthogonalization pass
        w = w - correction
    if norm(w) > tol:
        w = w / norm(w)
        append w to V
return V

```

于是，AlpsSweep 的离线阶段本质上是在构造一个“对全空间响应最有代表性”的低维试探空间。

11.3 ROM 矩阵是如何投影出来的

一旦有了基 V ，ROM 矩阵就通过 Galerkin 投影得到：

$$K_r = V^T K V, \quad M_r = V^T M V, \quad \mathbf{m}_{v,r} = V^T \mathbf{m}_v. \quad (49)$$

为什么这样做是合理的？因为原系统的每一项都满足“左乘测试空间、右乘试探空间”的标准有限元变分结构：体积项是二次型 $\mathbf{x}^T K \mathbf{x}$ 与 $\mathbf{x}^T M \mathbf{x}$ ；端口项是低秩外积 $\mathbf{m}_v \mathbf{m}_v^T$ 。在 Galerkin 框架下，只要把未知场近似为

$$\mathbf{x}(\omega) \approx V \mathbf{x}_r(\omega), \quad (50)$$

并用同一个 V 左乘残差，就会自然得到小系统

$$V^T A(\omega) V \mathbf{x}_r(\omega) = V^T \mathbf{b}(\omega). \quad (51)$$

将 (43) 代入后，立刻得到

$$\left(K_r - k_0^2(\omega) M_r + \sum_{v=1}^{N_v} Y_v(\omega) \mathbf{m}_{v,r} \mathbf{m}_{v,r}^T \right) \mathbf{x}_r(\omega) = \mathbf{b}_r(\omega), \quad (52)$$

其中

$$\mathbf{b}_r(\omega) = 2 \sum_{v \in \mathcal{E}} Y_v(\omega) a_v^{\text{inc}} \mathbf{m}_{v,r}. \quad (53)$$

这说明 ROM 并不是“重新拟合”全系统，而是把同一仿射结构投影到了低维空间。

11.4 为什么端口项在 ROM 中仍保持秩 1

端口项在全空间是 $\mathbf{m}_v \mathbf{m}_v^T$ ，它的秩至多为 1。投影后：

$$V^T (\mathbf{m}_v \mathbf{m}_v^T) V = (V^T \mathbf{m}_v) (V^T \mathbf{m}_v)^T = \mathbf{m}_{v,r} \mathbf{m}_{v,r}^T. \quad (54)$$

所以端口项的低秩结构被完全保留，只是向量长度从全空间维数 N 变成了 ROM 维数 r 。这也是 AlpsSweep 的关键优势之一：频率变化时，只需更新标量系数 $Y_v(\omega)$ ，无需重装稀疏矩阵。

11.5 和 ALPS 文献中的矩阵观点对应起来

从文献角度看，[6, 7, 8] 中的 ALPS / AWE / MOR 方法，本质上都是把一个关于频率的矩阵函数写成“若干固定矩阵 + 标量函数”的形式，再围绕某个展开点构造低维近似。这里的 AlpsSweep 做法与其一致，只是实现上更贴近有限元工程：

1. 先把有限元问题拆成频率无关的 $\mathbf{K}, \mathbf{M}, \mathbf{m}_v$;
2. 再用 $\mathbf{A}_0^{-1}$ 驱动 Krylov 递推生成基;
3. 最后在小空间里重建每个频点的矩阵 (52)。

因此，所谓“构造 ROM 矩阵”，并不是凭空造一个新模型，而是对全空间仿射系统做严格的投影降维。

11.6 为何 buildAffineSystem() 足够

注意到 (52) 里所有需要的数据都已经由 buildAffineSystem() 提供：

- $\mathbf{K}$ 与 $\mathbf{M}$ ：决定体积刚度与质量；
- $\mathbf{m}_v$ ：决定每个虚拟端口的秩 1 耦合；
- $k_{c,v}^2$ ：决定 $Y_v(\omega)$ 的截止行为；
- a_v^{inc} 与 isExcitationMode：决定右端是否注入入射波。

换句话说，AlpsSweep 构造 ROM 所需的一切代数素材都已经在离线阶段被标准化了；剩下的工作只是“选基 + 投影 + 小矩阵扫频”。

因此，buildAffineSystem() 输出的不是某个单频点矩阵，而是一个可被 AlpsSweep 直接消耗的仿射算子集合。ROM 矩阵的构造公式最终就是 (49) 和 (52)。在数值上，这让大规模三维端口问题从“每个频点都解一次大稀疏系统”变成“离线一次分解 + 在线反复解小稠密系统”，这正是 ALPS 的加速来源。